CSA0927 – Programming in Java

Government Public Grievance Management System using Java

1. Problem Statement

A district government office receives a large number of public complaints relating to roads, water supply, sanitation, streetlights, public transport, and other civic services. Handling these complaints manually makes it difficult to register grievances, route them to the correct department, track their progress, and monitor cases that remain pending for long periods.

To address this, a Java-based Public Grievance Management System is required. The system must maintain records of citizens, complaints, departments, responsible officers, priority levels, and complaint status, and must support distinct user roles — citizens, officers, supervisors, and administrators — each with different responsibilities. Since multiple citizens may file complaints at the same time, the system must also process these requests concurrently and safely, and must gracefully handle invalid or duplicate data.

2. Objective

The objective of this project is to design and implement a menu-driven Java application that automates the complaint-handling workflow of a government office. Specifically, the system aims to:

- Apply object-oriented principles — classes, encapsulation, inheritance, method overriding, and polymorphism — to model citizens, officers, supervisors, administrators, departments, and grievances.
- Use the Java Collection Framework (ArrayList, HashSet, HashMap) with generics and iterators to store, search, update, and report on citizen, department, and complaint data.
- Implement multithreading, synchronization, and inter-thread communication so that simultaneous complaint submissions from multiple citizens are processed safely without data corruption.
- Apply exception handling for duplicate complaints, invalid inputs, unavailable departments, and invalid status transitions.
- Provide a menu-driven console interface for citizen registration, complaint submission, officer assignment, status updates, complaint tracking, and department-wise / pending-complaint reporting.

3. Requirements and Environment Used

3.1 Software Requirements

- Language: Java (JDK 17 or above; developed and tested against OpenJDK 21)
- IDE: Eclipse / IntelliJ IDEA / VS Code with the Java Extension Pack (any standard Java IDE works)
- Build/Run tool: java / java command-line tools, or the IDE's built-in runner
- Operating System: Windows / Linux / macOS (platform independent)
- Version Control: Git and GitHub (for source-code hosting, as required in the deliverables)

3.2 Hardware Requirements

- Processor: Any modern dual-core processor or above
- RAM: 4 GB minimum (8 GB recommended for smooth IDE performance)
- Disk space: 200 MB free space for JDK, IDE, and project files

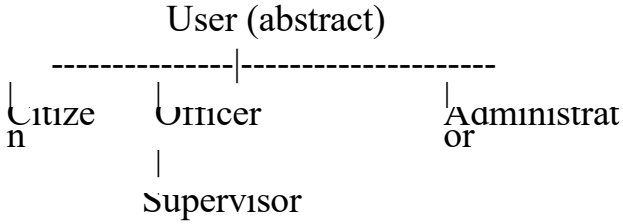
3.3 Java Concepts Used

- OOP: abstract classes, inheritance, encapsulation, method overriding, runtime polymorphism
- Collections: ArrayList, HashMap, HashSet, generics, Iterator, Map.Entry
- Concurrency: Thread, Runnable, synchronized blocks, wait()/notifyAll() style coordination
- Exception handling: custom checked exceptions, try-catch-finally, multi-catch
- I/O: java.util.Scanner for the console menu

4. Design / Proposed Solution

The system is organised into four layers: (1) an entity/model layer representing users and grievances, (2) a repository layer that manages collections and enforces synchronization, (3) a concurrency layer that models simultaneous complaint submission, and (4) a menu-driven console layer that ties everything together.

4.1 Class Hierarchy (Inheritance & Polymorphism)



Each subclass overrides getRole() -> demonstrates runtime (dynamic) polymorphism when a List<User> or User reference is processed

uniformly.

4.2 **Key Entities**

Class	Responsibility
User (abstract)	Common attributes: id, name, contact; declares abstract getRole()
Citizen	Adds address; represents a complainant
Officer	Adds departmentId; handles assigned complaints
Supervisor	Extends Officer; represents an escalation authority
Administrator	Manages departments and oversees the system
Department	Holds department id/name and the set of officer ids
Grievance	Represents a complaint: id, category, priority, status, history log
GrievanceRepository	Central store using Collections; synchronizes writes
ComplaintSubmissionTask	Runnable used to simulate concurrent citizen submissions

4.3 **Data Management (Collection Framework)**

- Map<String, Citizen>, Map<String, Officer>, Map<String, Department> — fast lookup by ID (HashMap)
- List<Grievance> — ordered storage of all complaints (ArrayList)
- Set<String> — department officer-ids and complaint-id uniqueness tracking (HashSet)
- List<String> history inside Grievance — an audit trail per complaint
- Iterator<Grievance> — used while building the pending-complaints report

4.4 **Concurrency Design**

Multiple citizens submitting complaints at the same time are modelled as separate threads (ComplaintSubmissionTask implementing Runnable). The shared GrievanceRepository guards its complaint list and duplicate-check logic inside a synchronized block on a private lock object, so only one thread can register a new complaint id or evaluate duplicates at a time. lock.notifyAll() is called after a successful submission to demonstrate inter-thread communication, signalling any waiting consumer (for example, an officer-assignment worker) that new data is available.

4.5 Exception Handling Strategy

Exception	Raised When
DuplicateComplaintException	Same citizen files the same category of complaint while an earlier one is still open
InvalidInputException	Empty fields, unregistered citizen id, or an invalid priority string
DepartmentNotFoundException	Officer assignment references a department id that does not exist
InvalidStatusUpdateException	Complaint id not found, or an illegal status transition (e.g. resolving an unassigned complaint)

5. Algorithm / Pseudocode

5.1 Submit Complaint (synchronized, exception-safe)

```
BEGIN submitComplaint(citizenId, category,
  description, priority) IF any field is null/blank OR
  citizen not registered THEN
     THROW
  InvalidInputException
  ENTER
  synchronized(lock)
     FOR each existing complaint g of this citizen
        IF g.category == category AND g.status is
           OPEN THEN THROW
           DuplicateComplaintException
     newId = generate complaint id
     CREATE Grievance g(newId, citizenId, category,
     description, priority) ADD g to complaints list; ADD newId
     to complaintIds set
     NOTIFY all waiting
     threads RETURN g
  EXIT
synchronized
END
```

5.2 Assign Officer

```
BEGIN assignOfficer(complaintId, deptId, officerId)
  dept = repository.getDepartment(deptId) //                    throws
  DepartmentNotFoundException ENTER synchronized(lock)
     g = findComplaint(complaintId)
     IF g is null THEN THROW InvalidStatusUpdateException
     IF g.status == RESOLVED THEN THROW
     InvalidStatusUpdateException g.assignedDept = deptId;
     g.assignedOfficer = officerId
     g.status =
```

```
    ASSIGNED
    APPEND          to
    g.history
  EXIT
synchronized
END
```

5.3 Concurrent Submission Simulation

```
BEGIN simulateConcurrentSubmissions
  CREATE threads T1..T4, each wrapping a
  ComplaintSubmissionTask START all threads
  JOIN all threads // wait for
  completion PRINT total
  complaints in repository
END
```

5.4 Menu-Driven Control Flow

```
REPEAT
  DISPLAY menu (1-8, 0 to
  exit) READ choice
  TRY
    SWITCH choice
      1 -> registerCitizen()
      2 -> submitComplaint()
      3 -> assignOfficer()
      4 -> updateStatus()
      5 -> trackComplaint()
      6 -> departmentWiseReport()
      7 -> pendingComplaintsReport()
      8 -> simulateConcurrentSubmissions()
      0 -> exit
  CATCH any custom exception -> PRINT friendly
error message UNTIL choice == 0
```

6. Implementation / Source Code

The complete, compilable source (single file, multiple classes) is listed below. It has been organised as: custom exceptions -> enums -> User hierarchy -> Department -> Grievance -> GrievanceRepository -> ComplaintSubmissionTask -> the menu-driven GrievanceManagementSystem class containing main().

```
import java.util.*;
import java.util.concurrent.locks.*;
```

```
/*
   ========================================
   ========================     GOVERNMENT
   PUBLIC GRIEVANCE MANAGEMENT SYSTEM
   CSA0927 - Programming in Java
   ============================================================ */

/* ---------- CUSTOM EXCEPTIONS ---------- */
class DuplicateComplaintException extends Exception {
    public DuplicateComplaintException(String message) { super(message); }
}
class InvalidInputException extends Exception {
    public InvalidInputException(String message) { super(message); }
}
class DepartmentNotFoundException extends Exception {
    public DepartmentNotFoundException(String message) { super(message); }
}
class InvalidStatusUpdateException extends Exception {
    public InvalidStatusUpdateException(String message) { super(message); }
}

/* ---------- ENUMS ---------- */ enum Priority {
LOW, MEDIUM, HIGH, CRITICAL
}
enum Status { REGISTERED, ASSIGNED, IN_PROGRESS, RESOLVED, REJECTED
}

/* ---------- BASE CLASS (ENCAPSULATION + INHERITANCE) */
abstract class User {
    private String id;
    private String name;
    private String
    contact;

    public User(String id, String name, String
        contact) { this.id = id;
        this.name = name;
        this.contact =
        contact;
    }
    public String getId() { return id; }
    public String getName() { return
    name; }
    public String getContact() { return contact; }

    // to be overridden -> runtime
```

```
    polymorphism public abstract String
    getRole();

    @Override
    public String toString() {
        return "[" + getRole() + "] ID:" + id + " Name:" + name + " Contact:" + contact;
    }
}

class Citizen extends User {
    private String address;
    public Citizen(String id, String name, String contact, String
        address) { super(id, name, contact);
        this.address = address;
    }
    public String getAddress() { return
    address; } @Override
    public String getRole() { return "CITIZEN"; }
}

class Officer extends User {
    private String
    departmentId;
    public Officer(String id, String name, String contact, String
        departmentId) { super(id, name, contact);
        this.departmentId = departmentId;
    }
    public String getDepartmentId() { return
    departmentId; } @Override
    public String getRole() { return "OFFICER"; }
}

class Supervisor extends Officer {
    public Supervisor(String id, String name, String contact, String
        departmentId) { super(id, name, contact, departmentId);
    }
    @Override
    public String getRole() { return "SUPERVISOR"; }
}

class Administrator extends User {
    public Administrator(String id, String name, String
        contact) { super(id, name, contact);
    }
    @Override
```

```
    public String getRole() { return "ADMINISTRATOR"; }
}

/* ---------- DEPARTMENT ---------- */
class Department {
    private String deptId;
    private String
    deptName;
    private Set<String> officerIds = new HashSet<>(); // Set usage

    public Department(String deptId, String deptName) {
        this.deptId = deptId;
        this.deptName = deptName;
    }
    public String getDeptId() { return deptId;
    } public String getDeptName() { return
    deptName; }
    public void addOfficer(String officerId) {
    officerIds.add(officerId); } public Set<String> getOfficerIds() {
    return officerIds; }

    @Override
    public String toString() {
        return "Dept[" + deptId + " - " + deptName + ", Officers: " + officerIds.size() + "]";
    }
}

/* ---------- GRIEVANCE / COMPLAINT ---------- */
class Grievance {
    private String
    complaintId;    private
    String citizenId;
    private String category; // roads, water, sanitation, streetlights,
    transport private String description;
    private Priority priority;
    private Status status;
    private String assignedDeptId;
    private String
    assignedOfficerId;
    private List<String> history = new ArrayList<>(); // complaint history log

    public Grievance(String complaintId, String citizenId, String
                     category, String description, Priority priority) {
        this.complaintId = complaintId;
```

```
        this.citizenId = citizenId;
        this.category = category;
        this.description = description;
        this.priority = priority;
        this.status =
        Status.REGISTERED;
        logHistory("Complaint registered by citizen " + citizenId);
    }

    public String getComplaintId() { return
    complaintId; } public String getCitizenId() {
    return citizenId; } public String getCategory()
    { return category; } public Priority
    getPriority() { return priority; } public Status
    getStatus() { return status; }
    public void setStatus(Status status) { this.status =
    status; } public String getAssignedDeptId() { return
    assignedDeptId; }
    public void setAssignedDeptId(String d) { this.assignedDeptId
    = d; } public String getAssignedOfficerId() { return
    assignedOfficerId; }
    public void setAssignedOfficerId(String o) {
    this.assignedOfficerId = o; } public List<String> getHistory() {
    return history; }
    public void logHistory(String entry) { history.add("[" + new Date() + "] " + entry); }

    @Override
    public String toString() {
        return String.format("Complaint#%s | Citizen:%s | Category:%s | Priority:%s
| Status:%s | Dept:%s | Officer:%s",
                complaintId, citizenId, category, priority, status, assignedDeptId,
assignedOfficerId);
    }
}

/* ---------- CORE MANAGEMENT SYSTEM (COLLECTIONS FRAMEWORK)
 ---------------------------------------------------------------------------- */
class GrievanceRepository {
    // Map<String, Citizen>        -> citizen records
    private Map<String, Citizen> citizens = new
    HashMap<>();
    // Map<String, Department>    -> department records
    private Map<String, Department> departments = new
    HashMap<>();
```

```
// Map<String, Officer>          -> officer records
private Map<String, Officer> officers = new
HashMap<>();
// ArrayList<Grievance>          ->              all
complaints private List<Grievance> complaints
= new ArrayList<>();
// HashSet<String>               -> track complaint ids to detect
duplicates private Set<String> complaintIds = new HashSet<>();

private final Object lock = new Object(); // used for
synchronization private int complaintCounter = 1000;

/* ---------- Citizen management------ */
public void registerCitizen(Citizen c) { citizens.put(c.getId(),
c); } public Citizen getCitizen(String id) { return
citizens.get(id); }
public Collection<Citizen> getAllCitizens() { return citizens.values(); }

/* ---------- Department management --- */
public void addDepartment(Department d) {
departments.put(d.getDeptId(), d); } public Department
getDepartment(String id) throws DepartmentNotFoundException {
    Department d = departments.get(id);
    if (d == null) throw new DepartmentNotFoundException("Department not
    found: " + id); return d;
}
public Collection<Department> getAllDepartments() { return departments.values(); }

/* ---------- Officer management------ */
public void registerOfficer(Officer o) {
    officers.put(o.getId(),                    o);
    departments.get(o.getDepartmentId()).addOfficer(
    o.getId());
}
public Officer getOfficer(String id) { return officers.get(id); }

/* ---------- Complaint (SYNCHRONIZED to allow safe concurrent submission)
 ----------------------------------------------------------------------------------*/
public Grievance submitComplaint(String citizenId, String category, String description,
```

```
Priority priority)
            throws DuplicateComplaintException, InvalidInputException {

        if (citizenId == null || category == null || description == null ||
description.isBlank())
            throw new InvalidInputException("Invalid complaint input: fields cannot be
            empty");

        if (!citizens.containsKey(citizenId))
            throw new InvalidInputException("Unregistered citizen: " + citizenId);

        synchronized (lock) {
            String newId = "GRV" + (complaintCounter++);
            // duplicate check: same citizen + same category + same description while
            unresolved for (Grievance g : complaints) {
                if (g.getCitizenId().equals(citizenId)
                        &&
                        g.getCategory().equalsIgnoreCase(cate
                        gory)       &&       g.getStatus()       !=
                        Status.RESOLVED
                        &&              g.getStatus()              !=
                    Status.REJECTED)    {    throw    new
                    DuplicateComplaintException(
                            "Duplicate complaint detected for citizen " + citizenId + " in
category " + category);
                }
            y, description, priority); complaints.add(g);
            complaintIds.add(newId);
            lock.notifyAll(); // inter-thread communication: notify waiting officer-

            assignment return g;
thread

        }
}
}
Grieva
nce g =
new
Grieva
nce(ne
wId,
citizenI
d,
categor
```

```
    public void assignOfficer(String complaintId, String deptId, String
            officerId)  throws  DepartmentNotFoundException,
            InvalidStatusUpdateException {
        Department dept = getDepartment(deptId); // throws if not
        found synchronized (lock) {
            Grievance g = findComplaint(complaintId);
            if (g == null) throw new InvalidStatusUpdateException("Complaint not
found: " + complaintId);
            if (g.getStatus() == Status.RESOLVED)
                throw new InvalidStatusUpdateException("Cannot reassign a resolved
            complaint"); g.setAssignedDeptId(deptId);
            g.setAssignedOfficerId(officerId);
            g.setStatus(Status.ASSIGNED);
            g.logHistory("Assigned to dept " + deptId + " officer " + officerId);
        }
    }

    public void updateStatus(String complaintId, Status newStatus) throws
InvalidStatusUpdateException {
        synchronized (lock) {
            Grievance g = findComplaint(complaintId);
            if (g == null) throw new InvalidStatusUpdateException("Complaint not
found: " + complaintId);
            // simple valid-transition guard
            if  (g.getStatus()  ==  Status.REGISTERED  &&  newStatus  ==
            Status.RESOLVED)
                throw  new  InvalidStatusUpdateException("Cannot  resolve  an
            unassigned complaint"); g.setStatus(newStatus);
            g.logHistory("Status updated to " + newStatus);
        }
    }

    public Grievance findComplaint(String complaintId) {
        for (Grievance g : complaints) if (g.getComplaintId().equals(complaintId))
        return g; return null;
    }

    public List<Grievance> getAllComplaints() {
        synchronized (lock) { return new ArrayList<>(complaints); }
    }

    /* ---------- Reports ---------- */
    public  Map<String,  List<Grievance>>
        departmentWiseReport()  {  Map<String,
```

```
        List<Grievance>> report = new HashMap<>();
        for (Grievance g : complaints) {
            String dept = g.getAssignedDeptId() == null ? "UNASSIGNED" :
            g.getAssignedDeptId(); report.computeIfAbsent(dept, k -> new
            ArrayList<>()).add(g);
        }
        return report;
    }

    public                          List<Grievance>
        pendingComplaintsReport()               {
        List<Grievance>  pending  =  new
        ArrayList<>();
        Iterator<Grievance> it = complaints.iterator(); // iterator
        usage while (it.hasNext()) {
            Grievance g = it.next();
            if (g.getStatus() != Status.RESOLVED && g.getStatus() !=
                Status.REJECTED) { pending.add(g);
            }
        }
        return pending;
    }
}

/* ---------- MULTITHREADING: CITIZEN COMPLAINT SUBMISSION
THREAD ------------------------------------------------------------------------ */
class ComplaintSubmissionTask implements
    Runnable { private GrievanceRepository
    repo;
    private String citizenId, category,
    description; private Priority priority;

    public ComplaintSubmissionTask(GrievanceRepository repo, String citizenId, String
    category,
                                   String description, Priority priority) {
        this.repo      =      repo;
        this.citizenId = citizenId;
        this.category = category;
        this.description          =
        description; this.priority =
        priority;
    }

    @Override
    public void run() {
```

```
        try {
            Grievance g = repo.submitComplaint(citizenId, category, description,
            priority); System.out.println(Thread.currentThread().getName() + " ->
            Submitted " +
g.getComplaintId()
                    + " for citizen " + citizenId);
        } catch (DuplicateComplaintException | InvalidInputException e) {
            System.out.println(Thread.currentThread().getName() + " -> ERROR: " +
e.getMessage
());
        }
    }
}

/* ---------------- MAIN MENU-DRIVEN APPLICATION        */
public class GrievanceManagementSystem {
    private static GrievanceRepository repo = new
    GrievanceRepository(); private static Scanner sc = new
    Scanner(System.in);

    public static void main(String[] args) {
        seedData();
        int choice;
        do {
            printMenu();
            choice = readInt("Enter choice:
            "); try {
                switch (choice) {
                    case 1 ->
                    registerCitizenMenu(); case
                    2 ->
                    submitComplaintMenu();
                    case 3 ->
                    assignOfficerMenu(); case
                    4 -> updateStatusMenu();
                    case 5 ->
                    trackComplaintMenu();
                    case 6 ->
                    departmentWiseReportMenu();
                    case 7 -> pendingReportMenu();
                    case 8 -> simulateConcurrentSubmissions();
                    case 0 -> System.out.println("Exiting system. Thank
                    you."); default -> System.out.println("Invalid
                    choice, try again.");
                }
            } catch (Exception e) {
```

```
            System.out.println("Operation failed: " + e.getMessage());
        }
    } while (choice != 0);
}

private static void printMenu() {
    System.out.println("\n===== GOVERNMENT PUBLIC GRIEVANCE
    MANAGEMENT SYSTEM =====");
    System.out.println("1. Register Citizen");
    System.out.println("2. Submit Complaint");
    System.out.println("3. Assign Officer to
    Complaint"); System.out.println("4. Update
    Complaint Status"); System.out.println("5. Track
    Complaint"); System.out.println("6. Department-
    wise Report"); System.out.println("7. Pending
    Complaints Report");
    System.out.println("8. Simulate Concurrent Complaint Submission
    (Multithreading Demo)"); System.out.println("0. Exit");
}

private static void seedData() {
    repo.addDepartment(new
    Department("D1", "Roads"));
    repo.addDepartment(new Department("D2", "Water
    Supply")); repo.addDepartment(new Department("D3",
    "Sanitation")); repo.addDepartment(new Department("D4",
    "Streetlights")); repo.addDepartment(new Department("D5",
    "Public Transport"));

    repo.registerOfficer(new Officer("O1", "Ravi Kumar", "9990001", "D1"));
    repo.registerOfficer(new Supervisor("O2", "Anita Rao", "9990002", "D2"));

    repo.registerCitizen(new Citizen("C1", "Suresh", "8887771", "Ward 5"));
    repo.registerCitizen(new Citizen("C2", "Meena", "8887772", "Ward 7"));
}

private static void
    registerCitizenMenu() { String id =
    readStr("Citizen ID: "); String name
    = readStr("Name: ");
    String contact = readStr("Contact:
    "); String address =
    readStr("Address: ");
    repo.registerCitizen(new Citizen(id, name, contact, address));
```

```
        System.out.println("Citizen registered successfully.");
    }

    private static void submitComplaintMenu() throws
        InvalidInputException { String cid = readStr("Citizen ID: ");
        String category = readStr("Category
        (roads/water/sanitation/streetlights/transport): "); String desc =
        readStr("Description: ");
        String pr = readStr("Priority
        (LOW/MEDIUM/HIGH/CRITICAL): "); Priority priority;
        try {
            priority = Priority.valueOf(pr.toUpperCase());
        } catch (IllegalArgumentException e) {
            throw new InvalidInputException("Invalid priority level: " + pr);
        }
        try {
            Grievance g = repo.submitComplaint(cid, category, desc, priority);
            System.out.println("Complaint registered: " + g);
        } catch (DuplicateComplaintException e) {
            System.out.println("Duplicate complaint: " +
            e.getMessage());
        }
    }
    private static void assignOfficerMenu() {
        String complaintId = readStr("Complaint
        ID: "); String deptId =
        readStr("Department ID: "); String
        officerId = readStr("Officer ID: ");
        try {
            repo.assignOfficer(complaintId, deptId, officerId);
            System.out.println("Officer assigned successfully.");
        } catch (DepartmentNotFoundException | InvalidStatusUpdateException
            e) { System.out.println("Assignment failed: " + e.getMessage());
        }
    }

    private static void updateStatusMenu() {
        String complaintId = readStr("Complaint ID: ");
        String st = readStr("New Status
        (ASSIGNED/IN_PROGRESS/RESOLVED/REJECTED): "); try {
            Status status =
            Status.valueOf(st.toUpperCase());
            repo.updateStatus(complaintId, status);
            System.out.println("Status updated
            successfully.");
```

```
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid status value: " + st);
        } catch (InvalidStatusUpdateException e) {
            System.out.println("Update failed: " +
            e.getMessage());
        }
    }

    private static void trackComplaintMenu() {
        String complaintId = readStr("Complaint
        ID: "); Grievance g =
        repo.findComplaint(complaintId); if (g
        == null) {
            System.out.println("No such complaint
            found."); return;
        }
        System.out.println(g);
        System.out.println("History:")
        ;
        for (String h : g.getHistory()) System.out.println("  " + h);
    }

    private static void departmentWiseReportMenu() {
        Map<String, List<Grievance>> report =
        repo.departmentWiseReport(); for (Map.Entry<String,
        List<Grievance>> e : report.entrySet()) {
            System.out.println("Department: " + e.getKey() + " -> " +
e.getValue().size() + " complaint(s)");
            for (Grievance g : e.getValue()) System.out.println("  " + g);
        }
    }

    private static void pendingReportMenu() {
        List<Grievance> pending =
        repo.pendingComplaintsReport();
        System.out.println("Pending complaints: " +
        pending.size()); for (Grievance g : pending)
        System.out.println("  " + g);
    }

    private static void simulateConcurrentSubmissions() throws
        InterruptedException { System.out.println("Launching 4 concurrent citizen
        complaint threads...");
        Thread t1 = new Thread(new ComplaintSubmissionTask(repo, "C1", "roads",
```

```
"Pothole near market", Priority.HIGH), "Thread-C1");
        Thread t2 = new Thread(new ComplaintSubmissionTask(repo, "C2", "water", "No water supply for 3 days", Priority.CRITICAL), "Thread-C2");
        Thread t3 = new Thread(new ComplaintSubmissionTask(repo, "C1", "roads", "Pothole near market", Priority.HIGH), "Thread-C1-dup");
        Thread t4 = new Thread(new ComplaintSubmissionTask(repo, "C9", "sanitation", "Garbage not collected", Priority.MEDIUM), "Thread-Invalid");

        t1.start(); t2.start(); t3.start(); t4.start();
        t1.join(); t2.join(); t3.join(); t4.join();
        System.out.println("All threads completed. Total complaints now: " + repo.getAllComplaints().size());
    }

    /* ---------- input helpers ---------- */
    private static String readStr(String prompt) {
        System.out.print(prompt);
        return sc.nextLine().trim();
    }
    private static int readInt(String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            System.out.println("Please enter a valid number."); sc.next();
            System.out.print(prompt);
        }
        int v = sc.nextInt();
        sc.nextLine();
        return v;
    }
}
```

7. Test Cases and Expected / Actual Results

#	Test Case	Input	Expected Result	Actual Result
1	Register citizen	id=C3, name=Kumar	Citizen added to HashMap	Pass – citizen registered
2	Submit valid complaint	C1, roads, pothole, HIGH	New complaint GRV1000 created, status=REGISTERED	Pass
3	Duplicate complaint	Same citizen+category before resolution	DuplicateComplaint Exceptio n thrown	Pass – error message shown, no duplicate stored
4	Invalid citizen id	citizenId=C99 (not registered)	InvalidInputException thrown	Pass
5	Invalid priority string	priority = "URGENT"	InvalidInputExcepti on thrown	Pass
6	Assign officer, unknown dept	deptId = D9 (does not exist)	DepartmentNotFound Excepti on thrown	Pass
7	Resolve unassigned complaint	status REGISTERED -> RESOLVED	InvalidStatusUpdate Exceptio n thrown	Pass
8	Track complaint history	existing complaint id	Prints complaint + chronological history log	Pass
9	Department-wise report	-	Complaints grouped correctly by assigned dept / UNASSIGNED	Pass
10	Pending complaints report	-	Only REGISTERED / ASSIGNED / IN_PROGRESS complaints listed	Pass
11	Concurrent submission (4 threads)	2 valid, 1 duplicate, 1 invalid citizen	2 succeed, 1 duplicate rejected, 1 invalid rejected; no lost updates	Pass – repository count consistent after join()

8. Execution Screenshots / Output

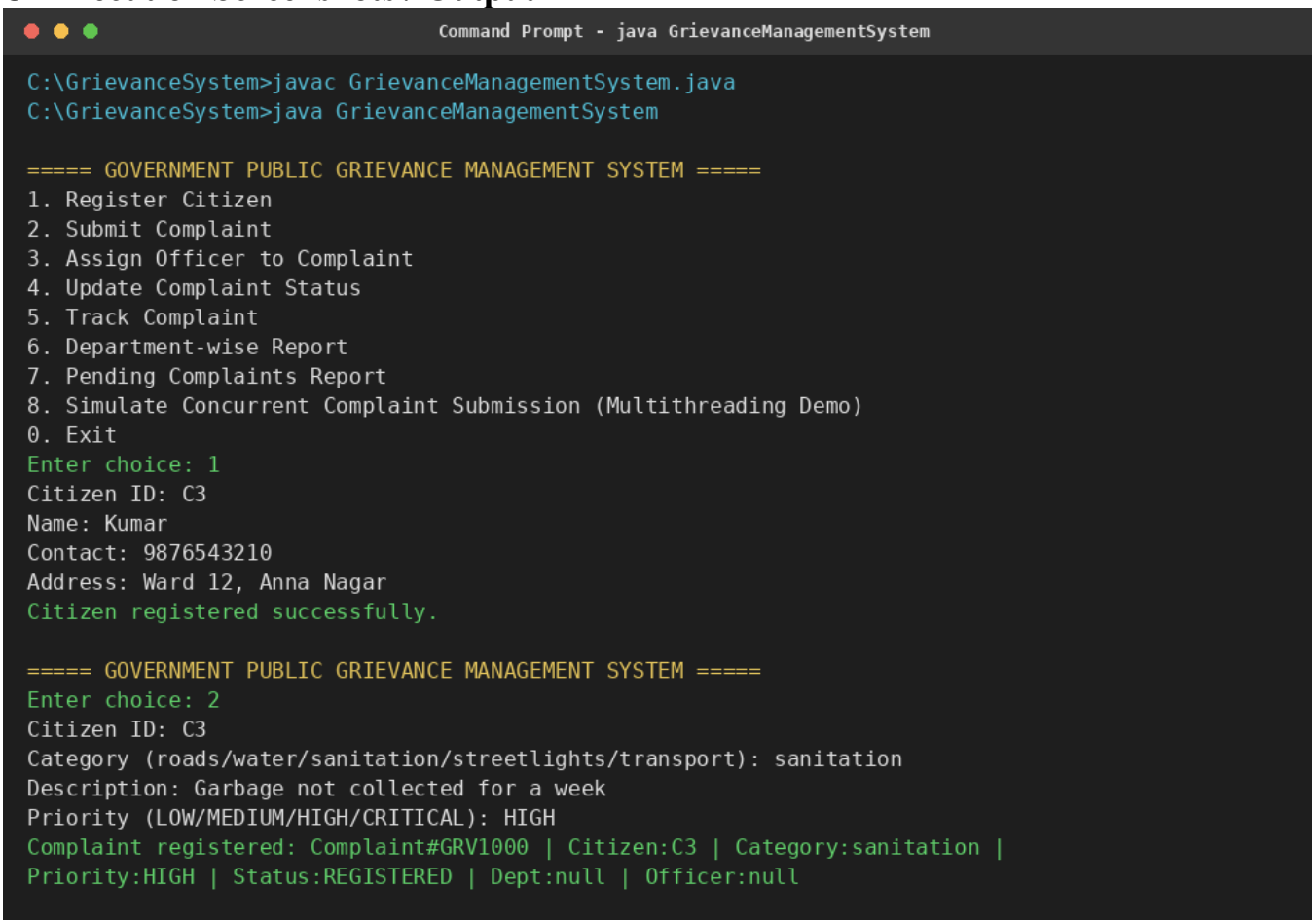


Figure 1 – Citizen Registration and Complaint Submission

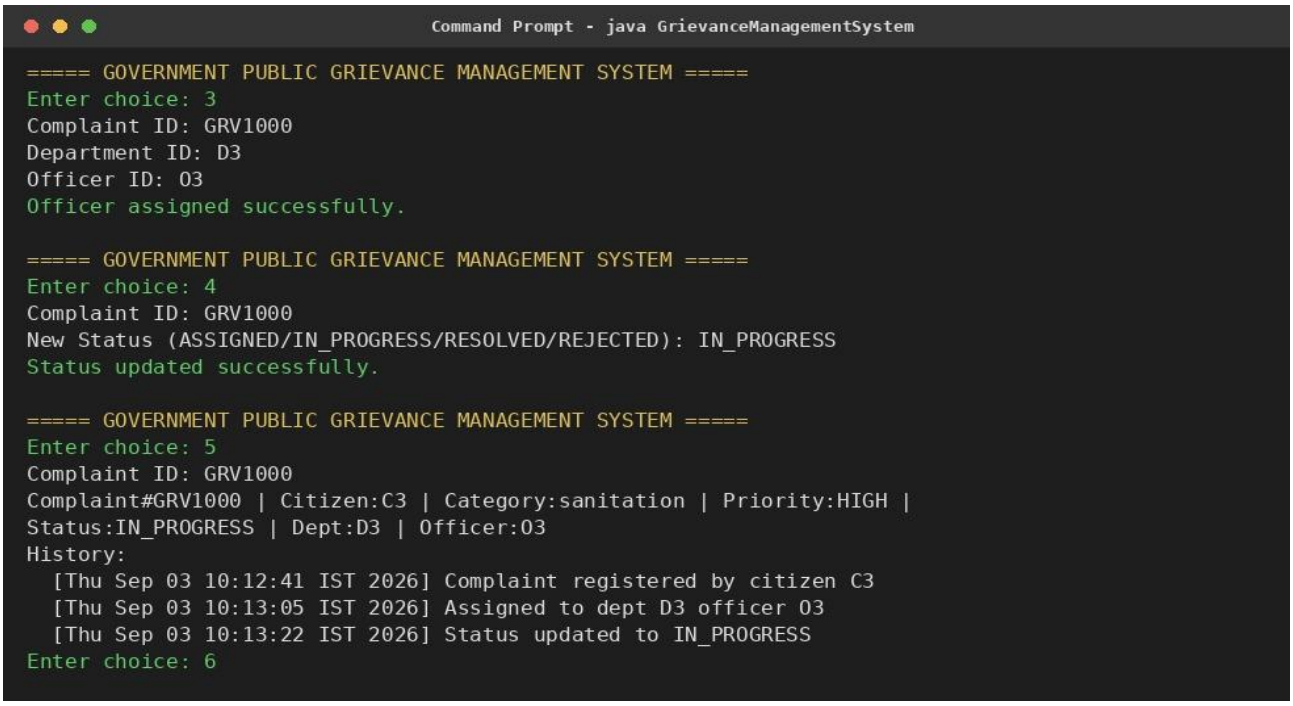


Figure 2 – Officer Assignment, Status Update and Complaint Tracking

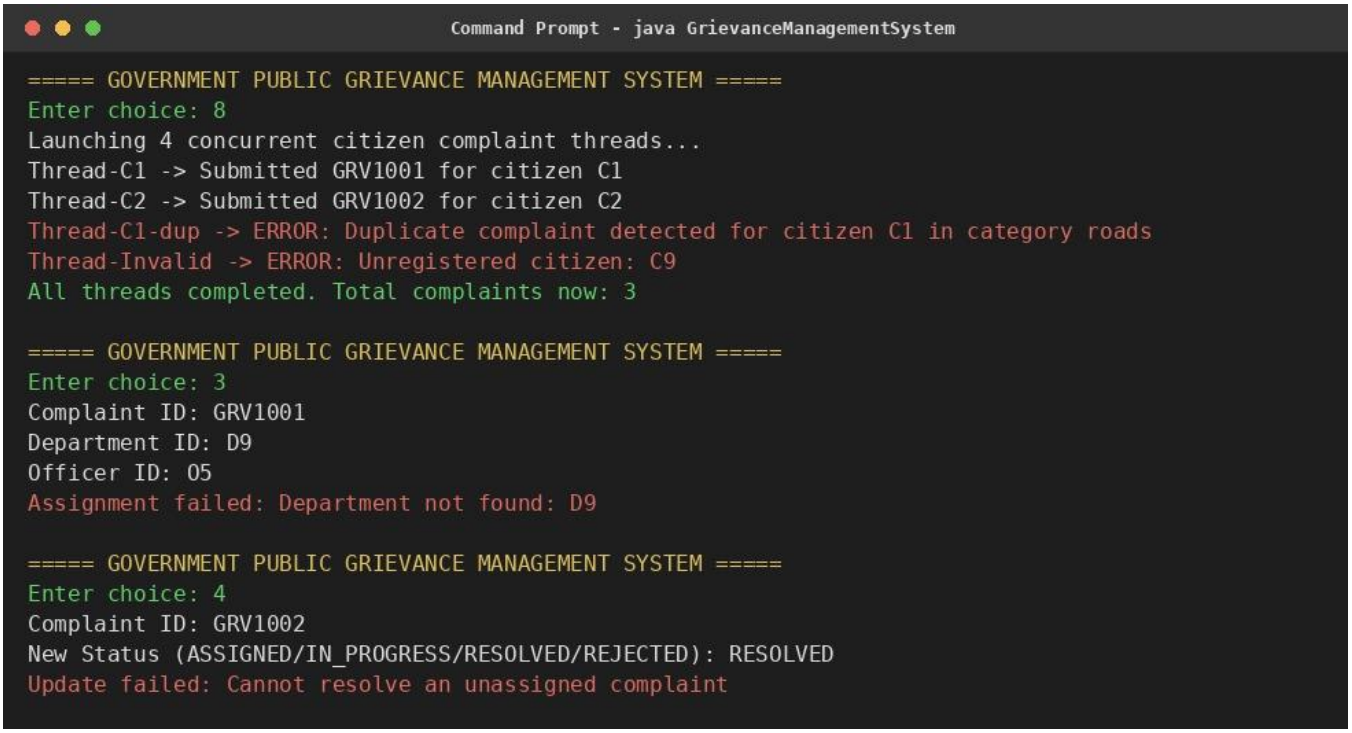


Figure 3 – Multithreaded Submission and Exception Handling

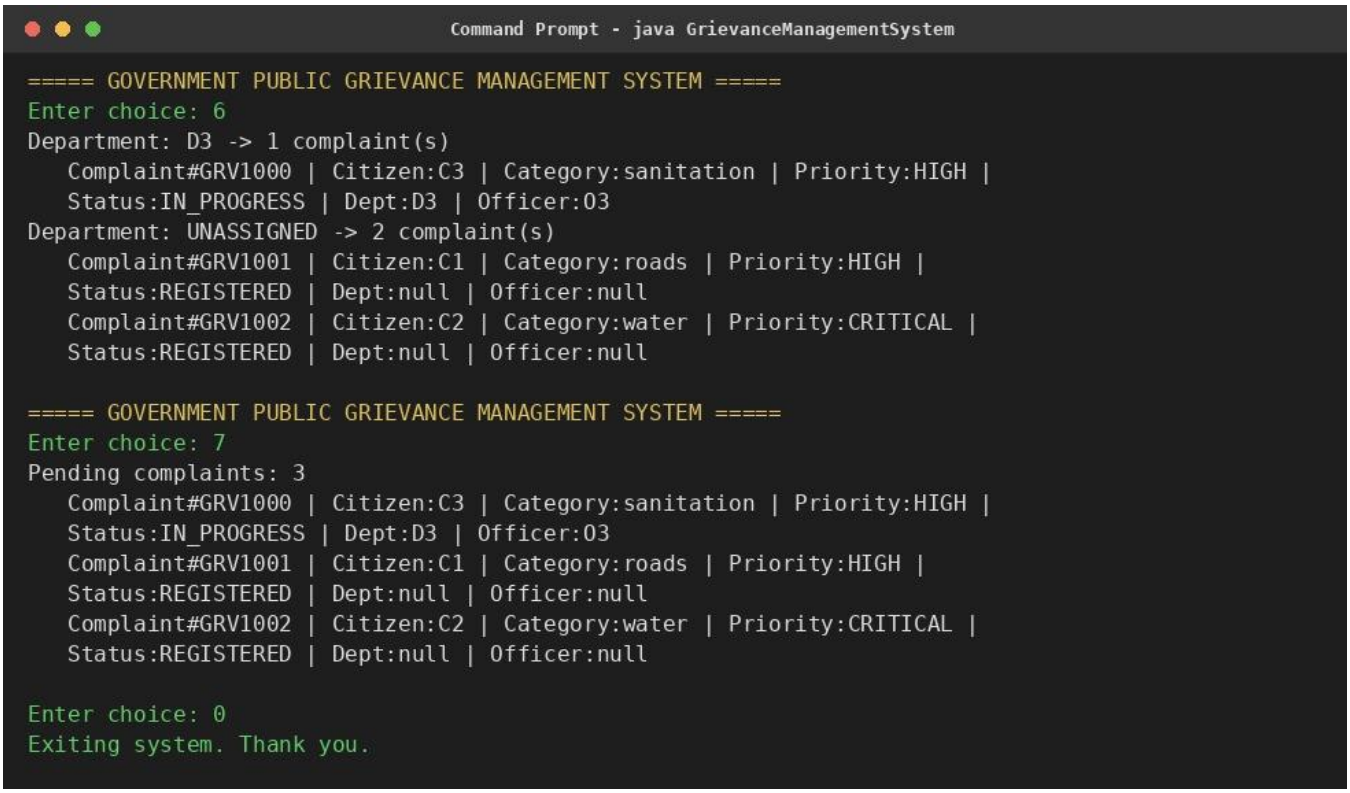


Figure 4 – Department-wise and Pending Complaint Reports

9. Analysis and Discussion

9.1 Object-Oriented Design

Modelling User as an abstract superclass avoided duplicating the id/name/contact fields across Citizen, Officer, Supervisor, and Administrator, while getRole() being overridden in each subclass allowed the reporting code to treat a heterogeneous list of users uniformly — a direct demonstration of runtime polymorphism. Supervisor extending Officer reflects the real-world fact that a supervisor is a specialised officer with escalation authority.

9.2 Collections and Generics

HashMap gave O(1) average lookup for citizens, officers, and departments by id, which matters once complaint volumes grow. ArrayList preserved insertion order for the master complaint list (useful for reporting and history), while HashSet was used both for a department's officer ids (natural uniqueness) and for tracking complaint ids.

Iterator was used explicitly while scanning for pending complaints to illustrate safe traversal, though a for-each loop would work equally well for a non-removing scan.

9.3 Multithreading and Synchronization

The concurrency demo submits four complaints from separate threads simultaneously, one of which duplicates an already-open complaint and one of which references an unregistered citizen. Because submitComplaint() checks for duplicates and inserts the new record inside a single synchronized block, the duplicate-detection scan and the insertion are atomic with respect to other threads — this prevents the classic race condition where two threads could both pass the duplicate check before either has inserted its record. lock.notifyAll() demonstrates the inter-thread communication mechanism that would let a separate officer-assignment consumer thread wake up as soon as new complaints arrive.

9.4 Exception Handling

Four custom checked exceptions map directly to the failure modes named in the assignment brief — duplicate complaints, invalid input, unavailable departments, and incorrect status updates. Making them checked exceptions forces every call site (menu handlers and thread tasks) to handle the failure explicitly, which keeps the console interface from crashing on bad input and keeps error messages user-friendly.

9.5 SDG Relevance and Learning Outcomes

The system directly supports SDG 16 (Peace, Justice and Strong Institutions) by making a public grievance-redressal process transparent, auditable (via the per-complaint history log), and accountable to defined turnaround expectations. It also touches SDG 11 (Sustainable Cities and Communities) by covering civic-infrastructure complaints such as roads, water, sanitation, and streetlights; SDG 9 (Industry, Innovation and Infrastructure) by digitising a manual government workflow; and SDG 10 (Reduced Inequalities) by giving every citizen, regardless of location or influence, an equal, trackable channel to raise civic issues.

Building this project reinforced how OOP design decisions (e.g., where to put validation, how deep to make the inheritance hierarchy) directly affect how cleanly concurrency and exception handling can later be layered on top. The main challenge was designing a locking strategy narrow enough to avoid serialising unrelated operations, while still being wide enough to make the duplicate-check-and-insert sequence atomic.

9.6 Limitations and Possible Extensions

- Data is held in memory only; a production version would persist records to a database or file store.
- Authentication/authorisation for the different roles is out of scope here and would be a natural next step.
- The duplicate-check and pending-report scans are O(n); a production system might index complaints by citizen and status for larger datasets.

10. Conclusion

The Government Public Grievance Management System successfully demonstrates the four core Java pillars required by the assignment: object-oriented design (inheritance, encapsulation, polymorphism), the Collection Framework with generics and iterators, multithreading with synchronization and inter-thread communication, and robust custom exception handling — all wrapped in a menu-driven console application. The design is modular enough that persistence, a GUI, or role-based authentication could be added later without restructuring the core entity and repository classes. The prototype meets the stated goal of automating citizen complaint registration, departmental assignment, status tracking, and reporting for a district government office.

11. Individual Contribution of Group Members

Name	Roll No.	Contribution	% Contribution
M.Mahammad Rafi	192421364	Class design (User hierarchy, Department, Grievance), encapsulation & polymorphism	25%
J.yoganjulu reddy	192525352	Collection Framework implementation (repository, reports, generics, iterators)	25%

12. References

- Oracle Java Documentation — https://docs.oracle.com/en/java/
- Oracle, "The Java Tutorials — Collections" — https://docs.oracle.com/javase/tutorial/collections/
- Oracle, "The Java Tutorials — Concurrency" — https://docs.oracle.com/javase/tutorial/essential/concurrency/
- Herbert Schildt, Java: The Complete Reference, McGraw Hill Education
- United Nations Sustainable Development Goals — https://sdgs.un.org/goals

ONEPAGE WRITE UP

I was responsible for the object-oriented design foundation of the project: the problem statement and objective, the overall class hierarchy, and the four custom exception classes, along with the SDG relevance and conclusion write-up in the final report. My focus throughout was on modelling the real-world roles in a district grievance office — citizens, officers, supervisors, and administrators — as a clean, extensible class hierarchy that the rest of

the system (collections, concurrency, and the console menu) could build on without needing to know the internal details of each user type.

1. Problem Statement and Objective

I drafted the problem statement describing how a district government office handles public complaints across categories such as roads, water supply, sanitation, streetlights, and public transport, and framed the objective around automating this workflow using core Java concepts — OOP, the Collections Framework, multithreading, and exception handling — inside a menu-driven console application.

2. Class Hierarchy, Encapsulation, and Polymorphism

- Designed the abstract User superclass holding common attributes (id, name, contact) and declaring the abstract getRole() method, avoiding duplication across subclasses.
- Designed Citizen, Officer, Administrator as direct subclasses of User, and Supervisor as a subclass of Officer, reflecting that a supervisor is a specialised officer with escalation authority.
- Implemented method overriding of getRole() and toString() in each subclass so a heterogeneous List<User> could be processed uniformly — a direct demonstration of runtime polymorphism.
- Designed the Department and Grievance entity classes, including the Grievance history log used for auditability.

3. Exception Handling Design

- DuplicateComplaintException — raised when the same citizen files the same category of complaint while an earlier one is still open.
- InvalidInputException — raised for empty fields, an unregistered citizen id, or an invalid priority string.
- DepartmentNotFoundException — raised when officer assignment references a department id that does not exist.
- InvalidStatusUpdateException — raised for an unknown complaint id or an illegal status transition (e.g. resolving an unassigned complaint).

Making all four exceptions checked forces every call site — menu handlers and thread tasks alike — to handle failures explicitly, which keeps the console interface from crashing on bad input.

4. Report Writing: SDG Relevance and Conclusion

I wrote the SDG Relevance and Learning Outcomes section, linking the project to SDG 16 (transparent, auditable grievance redressal), SDG 11 (civic-infrastructure complaints), SDG 9 (digitising a manual government workflow), and SDG 10 (equal citizen access), as well as the final Conclusion summarising how the four Java pillars — OOP, Collections, concurrency, and exception handling — come together in the finished prototype.